
实验三 基于 RISC-V 架构的 SM4-CBC 加密算法优化与性能分析

一、实验内容

本次实验旨在基于 RISC-V 架构，掌握 Linux 系统环境的搭建与 SM4-CBC 加密算法的优化方法。通过使用 RISC-V 工具链、构建 Linux 内核与根文件系统，并在 QEMU 中完成系统模拟运行；同时实现 SM4-CBC 算法，并通过多种优化策略进行性能分析，理解系统环境与优化手段对程序执行效率的影响。

本次实验需部署 QEMU 模拟器，由于编译过程耗时较长，请同学们务必按照实验指导书附录中的教程，提前完成实验环境的搭建工作，以确保实验顺利进行。

二、基本任务

1. 学习并掌握 RISC-V 工具链的基本使用方法，了解 riscv64-unknown-linux-gnu-编译器的功能。
2. 学习 risc-v Linux 内核的编译，使用 busybox 制作 linux 根目录，通过 OpenSBI 生成 linux 固件。并且使用 qemu 模拟器运行 risc-v Linux。
3. 编写 SM4-CBC 算法，并对其进行优化以降低算法的运行时间，可以考虑的优化策略包括但不限于：多线程，查表优化等。在实验设计中尽量考虑输入线程数、硬件配置（例如处理器核数、cache 大小）等各种你认为对运行时间会产生影响的参数。

三、考察重点

1. SM4-CBC 加密算法性能优化方法
2. 实验设计、数据统计方法与分析结论

四、注意事项

1. 实验报告需交代清楚实验的硬件软件环境，推荐 Ubuntu 22.04 LTS 或更新版本操作系统，Windows 环境下可使用 VMware 虚拟机装载 Ubuntu 系统。

五、实验报告要求

1. **4 人一组完成**，提交一份实验报告；
2. 提交形式为 .zip 压缩文件，命名方式为“**张三-李四-王五-赵六-实验三报告.pdf**”（命名方式不正确会扣分）。压缩包中需包括修改后的代码文件和实验报告文件，实验报告格式为 word 或 pdf，并在报告开头写上全组成员名字学

号。发送到山大网盘：

<https://icloud.sdu.edu.cn/link/AAED0F09115C054B018E140A6CB8F9E2E7>

3. 提交报告的截止时间为 **5 月 21 日**（周四晚上 11 点 59 分 59 秒）。晚交的报告将会被扣分。

4. 所有雷同的实验报告（包括和之前高年级提交的实验报告雷同），本次实验不得分。**同一学生本学期出现 2 次及以上雷同的实验报告，本学期实验为 0 分。**

5. 严格遵守实验室的规章制度，同时请爱护实验器材，如有器材损坏第一时间报告助教。

附录：实验环境配置（ubuntu22.04）•

本实验需使用实验一配置的 RISC-V GNU Compiler Toolchain 环境，相关配置请参考实验一附 1。

1、安装依赖项：

```
sudo apt install autoconf automake autotools-dev curl libmpc-dev  
libmpfr-dev libgmp-dev gawk build-essential bison flex texinfo gperf libtool  
patchutils bc zlib1g-dev libexpat-dev git libncurses5-dev libncursesw5-dev
```

2、创建主工作文件夹并进入目录

```
mkdir ~/riscv64-linux  
cd ~/riscv64-linux
```

3、安装用于启动 riscv64 平台上的内核的模拟器 qemu

```
sudo apt install qemu-system-misc
```

4、获取 Linux 源码并编译 Linux 内核

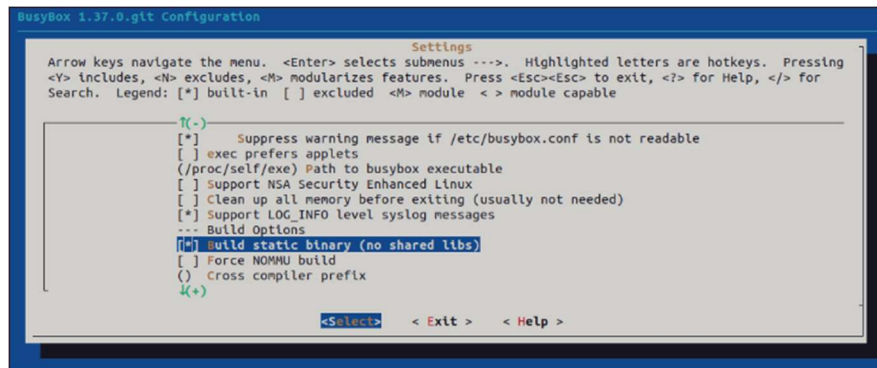
```
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-  
6.14.1.tar.xz
```

```
tar -xvf linux-6.14.1.tar.xz  
cd linux-6.14.1  
make ARCH=riscv CROSS_COMPILE=riscv64-unknown-linux-gnu- defconfig  
make ARCH=riscv CROSS_COMPILE=riscv64-unknown-linux-gnu- -j$(nproc)
```

5、获取并编译 busybox

```
cd ~/riscv64-linux  
git clone https://git.busybox.net/busybox  
cd busybox  
CROSS_COMPILE=riscv64-unknown-linux-gnu- make menuconfig
```

在弹出的菜单里找到 Settings -> Build static binary (no shared libs)，勾选：



然后 Exit -> Exit -> ... -> Yes

```
CROSS_COMPILE=riscv64-unknown-linux-gnu- make -j $(nproc)
```

```
CROSS_COMPILE=riscv64-unknown-linux-gnu- make install
```

如果遇到问题可以进行以下处理:

```
gedit .config
```

搜索 CONFIG_TC=y 改成 # CONFIG_TC is not set

```
CROSS_COMPILE=riscv64-unknown-linux-gnu- make oldconfig
```

```
CROSS_COMPILE=riscv64-unknown-linux-gnu- make -j $(nproc)
```

```
CROSS_COMPILE=riscv64-unknown-linux-gnu- make install
```

稍加等待,完成后生成_install 文件夹,这个文件夹下的东西就是根文件系统要用的。

6、获取并编译 OpenSBI

```
cd ~/riscv64-linux
```

```
git clone https://github.com/riscv-software-src/opensbi.git
```

```
cd opensbi/
```

```
git checkout v1.2
```

```
export CROSS_COMPILE=riscv64-unknown-linux-gnu-
```

```
make PLATFORM=generic
```

如果遇到问题可以进行以下处理:

```
make PLATFORM=generic CROSS_COMPILE=riscv64-unknown-linux-gnu-
```

```
PLATFORM_RISCV_XLEN=64 CC="riscv64-unknown-linux-gnu-gcc -std=gnu11"
```

编译完后，在 opensbi/build/platform/generic/firmware/目录下有生成的固件。本实验中采用 `fw_jump.elf`

7、自行编写 SM4 加密算法，并使用 RISC-V GNU Compiler Toolchain 进行编译

```
cd ~/riscv64-linux
riscv64-unknown-linux-gnu-gcc -static -pthread -o sm4 sm4.c
```

注：必须要加 `-static`，在编写多线程程序时要加 `-pthread`

8、制作一个最小的文件系统，并将写好的程序载入/tmp 中。

```
cd ~/riscv64-linux
qemu-img create rootfs.img 1g
mkfs.ext4 rootfs.img
```

`rootfs.img` 是文件系统的镜像文件名，1g 是磁盘文件大小，可以根据需要修改。并将磁盘文件格式化为 `ext4` 文件格式。

挂载这个 `img`，将 `_install` 目录下的东西拷贝到这个文件系统中，除此之外再创建一些必要的文件和目录。

```
mkdir rootfs
sudo mount -o loop rootfs.img rootfs
cd rootfs
sudo cp -r ../busybox/_install/* .
sudo mkdir proc sys dev tmp etc etc/init.d
```

然后另外再新建一个最简单的 `init` 的 `RC` 文件：

```
cd etc/init.d/
sudo vim rcS
```

该文件如下：

```
#!/bin/sh

mount -t proc none /proc
mount -t sysfs none /sys
/sbin/mdev -s
```

然后修改 `rcS` 文件权限，加上可执行权限，这样当 `busybox` 的 `init` 运行起来后，就能运行这个 `/etc/init.d/rcS` 脚本：

```
sudo chmod +x rcS
```

接下来将 5 中编写的程序拷贝入/tmp 中。并给他加上可执行权限：

```
sudo cp -r ~/riscv64-linux/sm4 ~/riscv64-linux/rootfs/tmp
sudo chmod +x ~/riscv64-linux/rootfs/tmp/sm4
```

最后退出 rootfs 目录并卸载文件系统：

```
cd ~/riscv64-linux
sudo umount rootfs
```

至此，文件系统就制作完成了。

注：接下来的实验中，如果需要更改自己所编写的程序，只需重新挂载 rootfs 与 rootfs.img，并将新程序拷贝入/tmp 中即可。

9、使用 qemu 运行 risc-v linux

```
cd ~/riscv64-linux
qemu-system-riscv64 -M virt -smp 8 -m 8192 \
    -bios opensbi/build/platform/generic/firmware/fw_jump.elf \
    -kernel linux-6.14.1/arch/riscv/boot/Image \
    -append "rootwait root=/dev/vda ro" \
    -drive file=rootfs.img,format=raw,id=hd0 \
    -device virtio-blk-device,drive=hd0 \
    -netdev user,id=net0 -device virtio-net-device,netdev=net0 -
```

nographic

成功运行后的效果如下：

```
eve@eve-virtual-machine:~/riscv64-linux$ qemu-system-riscv64 -M virt -smp 8 -m 8192 \
> -bios opensbi/build/platform/generic/firmware/fw_jump.elf \
> -kernel linux-6.14.1/arch/riscv/boot/Image \
> -append "rootwait root=/dev/vda ro" \
> -drive file=rootfs.img,format=raw,id=hd0 \
> -device virtio-blk-device,drive=hd0 \
> -netdev user,id=net0 -device virtio-net-device,netdev=net0 -nographic
OpenSBI v1.2
OpenSBI
Platform Name      : riscv-virtio,qemu
Platform Features  : mdeleg
Platform HART Count: 8
Platform IPI Device: aclint-msw
Platform Timer Device: aclint-mtimer @ 100000000Hz
Platform Console Device: uart8250
Platform HSM Device: ---
Platform PMU Device: ---
Platform Reboot Device: sifive_test
Platform Shutdown Device: sifive_test
Firmware Base      : 0x00000000
Firmware Size      : 264 KB
Runtime SBI Version: 1.0
Domain0 Name       : root
Domain0 Boot HART  : 0
Domain0 HARTS      : 0*,1*,2*,3*,4*,5*,6*,7*
Domain0 Region00   : 0x0000000000000000-0x000000000200ffff (I)
Domain0 Region01   : 0x0000000000000000-0x000000000007ffff (I)
Domain0 Region02   : 0x0000000000000000-0xffffffffffffffff (R,W,X)
Domain0 Next Address: 0x0000000002000000
Domain0 Next Arg1  : 0x0000000002200000
Domain0 Next Mode   : S-mode
Domain0 SysReset    : yes
```

注：输入 ctrl+a 后再输入 x 后即可退出 qemu

10、在 qemu 的 risc-v linux 中运行自己所编写的程序

```
cd /tmp
```

```
./sm4
```